

Pauta de Corrección

Interrogación 2 - IIC2513
Tecnologías y Aplicaciones Web

Pontificia Universidad Católica de Chile

19 de noviembre de 2025

Índice

1. Cookies, Session, JWT, OAuth [90 pts]	3
1.a. Diferencias entre mecanismos de autenticación [30 pts]	3
1.b. Diseño híbrido [20 pts]	4
1.c. Flujo de autenticación para /api/mis-cursos [40 pts]	4
2. API REST, HTTP, CRUD, Códigos de estado [90 pts]	5
2.a. Stateless en API REST [20 pts]	5
2.b. Diseño de rutas y verbos HTTP para CRUD [40 pts]	6
2.c. Status code y JSON para curso inexistente [30 pts]	6
3. MVC y SPA [40 pts]	6
3.a. Patrón MVC [20 pts]	7
3.b. SPA (Single Page Application) [20 pts]	7
4. DOM, JS, UX/UI, Render Tree [90 pts]	8
4.a. DOM, Event Listeners y Render Tree [40 pts]	8
4.b. Mejoras UX/UI y accesibilidad [20 pts]	9
4.c. DOM, CSSOM y Render Tree: diferencias y formación [30 pts]	10
5. bcrypt y Política de contraseñas [60 pts]	10
5.a. Conceptos de bcrypt [40 pts]	10
5.b. Política de contraseñas [20 pts]	12
6. Evaluación de stack tecnológico [60 pts]	12
6.a. Supuestos del sistema [20 pts]	12
6.b. Tecnologías para el backend [20 pts]	14
6.c. Framework de frontend basado en Python [20 pts]	15
7. React [90 pts]	16
7.a. Conceptos fundamentales [40 pts]	16
7.b. Comunicación padre-hijo con CursoCard [50 pts]	18
8. Vue.js [80 pts]	20
8.a. beforeCreate vs created [40 pts]	20
8.b. Reloj en onUpdated vs onMounted [40 pts]	22

9. Bonus 1: React (useEffect + Virtual DOM) [50 pts]	24
9.a. Estructura de useEffect para Fetch [15 pts]	24
9.b. Análisis de bucle infinito (Pseudocódigo) [15 pts]	25
10.Bonus 2: Vue 3 (ref + fetch + reactividad) [50 pts]	25
10.a. Conceptos de Reactividad [20 pts]	25
10.b. Proceso de carga de datos [20 pts]	26
10.c. Análisis de falla [10 pts]	26
11.Bonus 3: ORM [50 pts]	27
11.a. Conceptos de ORM y Mapeo [25 pts]	27
11.b. Flujo arquitectónico de creación [10 pts]	27
11.c. Errores de sincronización [15 pts]	28

Pregunta 1: Cookies, Session, JWT, OAuth [90 pts]

1.a Diferencias entre mecanismos de autenticación [30 pts]

Se espera que la respuesta compare los 3 mecanismos mencionados.

Cookies de sesión (Session ID + Sesión en servidor) [10 pts]

Almacenamiento [5 pts]:

- El estado completo de la sesión está en el servidor (puede ser en base de datos)
- En el cliente se guarda un identificador de sesión dentro de la cookie (en el browser)
- Cookie junto a la sesión: el servidor mantiene el estado

Qué viaja en cada request [5 pts]:

- En cada request viaja la cookie con el session ID
- El servidor usa ese ID para buscar la sesión en la BDD (u otro lugar de almacenamiento)

JWT [10 pts]

Almacenamiento [5 pts]:

- El estado (o cualquier otra información) va dentro del token
- El JWT es *stateless* con respecto a la sesión

Qué viaja en cada request [5 pts]:

- En cada request viaja el token JWT completo (`header.payload.signature`)

Nota: Verificar que no se confunda el concepto de cookie con el de sesión.

Riesgos y mitigaciones [10 pts]

Riesgos (mencionar al menos 2):

- Robo de cookies/tokens (XSS, robo de dispositivo, MITM sin HTTPS)
- Tokens/ID sin expiración razonable
- Almacenamiento inseguro (`localStorage` en vez de `sessionStorage`)

Mitigaciones [5 pts] (un par para cookies, otro par para JWT):

- Cookies con `HttpOnly`, `Secure`, `SameSite` (o mencionar políticas de seguridad)
- Usar HTTPS
- Expiración en cookies y JWT
- Uso de refresh tokens
- Listas negras y listas blancas

- En JWT: validar la firma y aumentar rounds de salt

1.b Diseño híbrido [20 pts]

Se espera una propuesta bien justificada en seguridad/escalabilidad.

Sesiones y cookies [10 pts]

Casos de uso:

- Web tradicional (Cliente-servidor, SPA, etc.) con browser en front, donde es necesario usar políticas de seguridad (`HttpOnly`, `SameSite`, etc.)
- Paneles de uso exclusivo de administrador (u otro rol con privilegios mayores), áreas sensibles donde se necesita control de sesión por servidor

JWT

Casos de uso:

- APIs REST consumidas por servicios externos
- Comunicación servicio a servicio (microservicios u otros)
- Situaciones donde la escalabilidad (muchos nodos) y el *stateless* sean relevantes

Justificación en seguridad y escalabilidad [10 pts]

Seguridad:

- Las cookies + sesión permiten validar/invalidar la sesión en el servidor al hacer login/logout
- En JWT es menos directo, pero se pueden usar tokens de corta duración y/o refresh tokens

Escalabilidad:

- Sesiones exigen almacenamiento compartido (BDD) si hay varios servidores
- JWT reduce carga de sesión en el servidor (solo firma/verificación)

1.c Flujo de autenticación para /api/mis-cursos [40 pts]

Se espera un paso a paso del middleware que valida JWT antes de llegar al controlador.

Pasos esperados:

1. Extraer el token:

- Leer de la cabecera `Authorization: Bearer <token>`
- Verificar que existe

2. Verificar token (firma):

- Usar biblioteca (ej: `jwt.verify`)
- Validar la firma con `secret`
- Si la firma no coincide, cancelar con error

Pregunta 2: API REST, HTTP, CRUD, Códigos de estado [90 pts]

2.a Stateless en API REST [20 pts]

Definición clara de "stateless"[10 pts]

Respuestas adecuadas:

- El servidor no guarda estado de sesión entre requests del cliente (no tiene memoria de conversación previa)
- El servidor trata cada request como independiente

Distribución de puntaje:

- **10 pts:** Define correctamente stateless (no mantener estado de sesión entre requests)
- **8 pts:** Idea central clara con imprecisiones menores
- **5 pts:** Menciona "no guarda información" sin conceptos claros
- **0 pts:** Definición equivocada (ej: confundir stateless con no usar BDD)

Impacto en el diseño del backend [10 pts]

Ideas esperadas:

- Cada request debe incluir credenciales/información de contexto
- El backend stateless es más fácil de escalar (cualquier nodo puede atender el request)
- Si se necesita estado de largo plazo, se guarda en BDD
- Mencionar diseño con tokens
- Mencionar manejo de expiración de cookies o duraciones largas
- Estrategias de almacenamiento local con tratamiento de seguridad

Distribución de puntaje:

- **10 pts:** Explica coherentemente efectos concretos en el diseño
- **8 pts:** Menciona efectos con explicaciones poco claras
- **3 pts:** Ideas en punteo sin justificación
- **0 pts:** No relaciona stateless con diseño del backend

2.b Diseño de rutas y verbos HTTP para CRUD [40 pts]

Se espera uso coherente de recursos con verbos correctos.

Endpoints esperados (10 pts c/u):

1. **Crear curso:** POST /cursos
2. **Listar todos los cursos:** GET /cursos
3. **Editar nombre del curso:**
 - PUT /cursos/{id} (curso completo)
 - PATCH /cursos/{id} (solo campo nombre)
 - Ambas opciones son válidas
4. **Eliminar curso:** DELETE /cursos/{id}

Criterios de evaluación:

- Usa /cursos y /cursos/{id} según corresponde
- Asigna correctamente los verbos HTTP
- **Descuentos por endpoint** si no usa {id} o usa verbos incorrectamente

2.c Status code y JSON para curso inexistente [30 pts]

Request: GET /cursos/999 (curso no existe)

Código de estado [15 pts]:

- **Código ideal:** 404 (Not Found)
- **Válido:** Cualquier código en rango 400s
- Si no hay justificación clara, restar 5 puntos

Estructura JSON mínima [15 pts]:

Respuesta JSON bien formada y coherente. Ejemplos:

```
{
  "error": "Curso no encontrado",
  "code": "COURSE_NOT_FOUND"
}

o

{
  "message": "Curso 999 no existe"
}
```

Pregunta 3: MVC y SPA [40 pts]

3.a Patrón MVC [20 pts]

Explicar el modelo de manera adecuada sin confundir los roles.

Distribución de puntaje: 7 pts (M) + 7 pts (C) + 6 pts (V)

Modelo:

- Acceso a bases de datos
- Recursos del sistema de archivos
- Reglas de negocio

Controlador:

- Recibe request del navegador
- Llama al Modelo para obtener/actualizar datos
- Coordina, pero no renderiza ni contiene lógica de negocio pesada

Vista (V):

- Es la interfaz que el usuario ve (el HTML, CSS, JS)
- Recibe los datos procesados por el Controlador
- ****Genera/Renderiza el HTML final**** usando plantillas (templates)
- No contiene lógica de negocio

Importante: No mezclar acciones del controlador con el servidor.

3.b SPA (Single Page Application) [20 pts]

Explicación [5 pts]

- **Solo siglas (2 pts):** Si solo menciona "Single Page Application"
- **Con explicación (5 pts):**
 - Aplicación de una sola página que carga una vez
 - Actualiza la vista vía JavaScript
 - Navegación client-side sin recarga completa
 - Usa APIs (REST) para traer datos dinámicamente

Ejemplo donde SPA es mejor que HTML estático [15 pts]

Justificación: Debe haber interacción constante, actualización parcial o mucha dinámica.

Ejemplos adecuados:

- Panel de administración con dashboards dinámicos
- Chat en tiempo real
- Correo web (Gmail)

- Sistemas con navegación rápida sin recargar (Trello, Kanban)
- Juegos (como en el proyecto)
- Contenido dinámico con muchos elementos (álbum Pokémon)

Distribución de puntaje:

- **15 pts:** Ejemplo concreto + explicación razonable y coherente
- **7 pts:** Ejemplo válido + justificación breve/incompleta
- **3 pts:** Solo ejemplo adecuado sin explicación
- **0 pts:** Mal ejemplo (ej: página estática)

Pregunta 4: DOM, JS, UX/UI, Render Tree [90 pts]

4.a DOM, Event Listeners y Render Tree [40 pts]

DOM [10 pts]

Conceptos esperados:

- Representa la página como un árbol de nodos
- El browser crea estructura de árbol
- Permite a JavaScript leer y modificar elementos dinámicamente

Distribución de puntaje:

- **10 pts:** Explica DOM como árbol editable en memoria del browser
- **5 pts:** Menciona árbol y manipulación (incompleto)
- **2 pts:** Menciona HTML sin aclarar representación en memoria
- **0 pts:** Concepto incorrecto

Event Listeners [10 pts]

Conceptos esperados:

- JavaScript/DOM usa listeners para reaccionar a eventos del usuario (click, input, key-press)
- Permiten conectar DOM con lógica de programación en JS

Distribución de puntaje:

- **10 pts:** Explica relación entre funciones y eventos del browser/DOM
- **6 pts:** Explicación parcial
- **3 pts:** Menciona eventos pero confunde conceptos
- **0 pts:** Incorrecto

Render Tree [10 pts]

Conceptos esperados:

- Fusión de DOM + CSSOM
- Permite visualizar estructura y estilo en pantalla
- Solo incluye nodos visibles

Distribución de puntaje:

- **10 pts:** Explica como fusión DOM/CSSOM y rol en renderizado
- **8 pts:** Comprende que ayuda a renderizar, sin explicar CSSOM o relación con DOM
- **5 pts:** Mención vaga o superficial
- **0 pts:** Incorrecto

Ventajas de separar HTML, CSS y JS [10 pts]

Mencionar 2 ventajas:

- Separación de responsabilidades (estructura/estilos/lógica)
- Mejor mantenimiento del proyecto
- Reutilización de código
- Buenas prácticas de programación

Distribución de puntaje:

- **10 pts:** 2 ventajas correctas y bien explicadas
- **8 pts:** 2 ventajas sin elaboración
- **5 pts:** Algo correcto pero vago, o solo 1 justificada
- **0 pts:** Incorrecto

4.b Mejoras UX/UI y accesibilidad [20 pts]

Propuestas de mejora [15 pts]

Expectativas: Reducir errores, mejorar claridad, consistencia, accesibilidad.

Cambio esperado: Nombres de botones de `Curso+` `Curso-`.^a `Agregar` `Eliminar`

Otros aspectos:

- Desactivar botón mientras se procesa
- Mostrar mensajes de feedback (`Curso agregado/eliminado`)
- Colores consistentes (verde=añadir, rojo=eliminar)

Distribución de puntaje:

- **15 pts:** 2 mejoras claras y bien justificadas
- **10 pts:** 2 mejoras válidas + poca justificación
- **8 pts:** 1 mejora clara + 1 vaga
- **7 pts:** 1 mejora justificada claramente
- **4 pts:** 1 mejora parcialmente justificada
- **0 pts:** Propuestas irrelevantes o incorrectas

Fuente oficial sobre accesibilidad [5 pts]

Respuesta correcta: W3C (también válido: WCAG sin mencionar W3C)
Distribución de puntaje:

- **5 pts:** Nombra W3C/WCAG/WAI
- **0 pts:** Blogs, Reddit, StackOverflow u otros no oficiales

4.c DOM, CSSOM y Render Tree: diferencias y formación [30 pts]

Conceptos esperados:

- **DOM:** Árbol del documento a partir del HTML
- **CSSOM:** Árbol que representa las reglas CSS aplicables
- **Render Tree:** Combinación de DOM + CSSOM en el browser

Proceso de carga:

1. Parser HTML crea DOM
2. Parser CSS crea CSSOM
3. Browser combina ambos en Render Tree
4. Render Tree se renderiza (pinta)

Distribución de puntaje:

- **30 pts:** Explica claramente los 3 árboles con rol y alcances
- **20 pts:** Buena explicación pero incompleta
- **10 pts:** Explicación superficial o moderadamente incompleta
- **0 pts:** Confunde conceptos, muy vaga o incompleta

Pregunta 5: bcrypt y Política de contraseñas [60 pts]

5.a Conceptos de bcrypt [40 pts]

bcrypt [10 pts]

Conceptos esperados: Función/biblioteca de hashing para contraseñas.

Distribución de puntaje:

- **10 pts:** Explica como hash seguro, menciona importancia de salt.rounds
- **8 pts:** Explica que sirve para hashear contraseñas
- **3 pts:** Dice cifra contraseñas”(incorrecto) pero da concepto
- **0 pts:** Entendimiento incorrecto

salt [10 pts]

Conceptos adecuados:

- Evita ataques con tablas rainbow
- Asegura que dos contraseñas iguales generen hashes distintos
- Evita comparación de hashes entre usuarios
- Está incorporado en el resultado de bcrypt

Distribución de puntaje:

- **10 pts:** Indica claramente que salt evita hashes repetidos
- **6 pts:** Explica que hace hashes distintos, sin detalle
- **3 pts:** Menciona salt de forma incompleta o confusa
- **0 pts:** Incorrecto (ej: .el salt cifra la contraseña”)

salt rounds y su impacto [10 pts]

Conceptos esperados:

- Salt rounds = número de iteraciones o factor de costo
- Aumenta el tiempo de cómputo del hash
- Más seguridad contra fuerza bruta
- Impacto en rendimiento del servidor

Distribución de puntaje:

- **10 pts:** Explica rounds como costo computacional, impacto en seguridad y rendimiento
- **6 pts:** Menciona que más rounds es más lento pero favorece seguridad
- **3 pts:** Respuesta superficial o incompleta
- **0 pts:** Incorrecto (ej: cuántas veces se encripta”)

Por qué el hash es irreversible [10 pts]

Concepto esperado:

- Por diseño: el hash no tiene función inversa matemática
- A diferencia del cifrado, que sí tiene clave para volver al original
- Usa funciones unidireccionales

Distribución de puntaje:

- **10 pts:** Explica irreversibilidad como parte del diseño del hash
- **5 pts:** Explicación poco justificada
- **0 pts:** Incorrecto

5.b Política de contraseñas [20 pts]

Expectativas: Política coherente y razonada con mínimo 3 criterios.

Criterios adecuados:

- Longitud mínima: 10-12 caracteres
- Variedad: mayúsculas/minúsculas, números, símbolos (opcional)
- Evitar información personal (nombre, RUT, fecha de nacimiento, mascota)
- Evitar contraseñas comunes ("123456", "password", "clave123")
- No reutilizar contraseñas anteriores
- Bloqueo o retardo tras intentos fallidos
- Tiempo de expiración razonable (opcional)
- Verificación de fortaleza (zxcvbn u otro)

Distribución de puntaje:

- **20 pts:** Política clara con criterios razonados (mínimo 3: longitud, passwords comunes, variedad)
- **12 pts:** Menos de 3 criterios o criterios débiles
- **4 pts:** Política incompleta o muy superficial
- **0 pts:** Políticas inseguras o incorrectas

Pregunta 6: Evaluación de stack tecnológico [60 pts]

Pregunta con supuestos laxos. Se evalúa coherencia global, buenos supuestos y conocimiento técnico.

6.a Supuestos del sistema [20 pts]

Aspectos a considerar

Aspectos a considerar:

- Cantidad de alumnos potenciales (decenas de miles)
- Usuarios concurrentes (fracción de alumnos potenciales)
- Seguridad (aplicación de conceptos de clase)
- Disponibilidad (24x7, salvo justificación)
- Criticidad (media)
- Confidencialidad (alta: datos personales, RUT, calificaciones, horarios)

Coherencia: No importa el número exacto, sino la coherencia. Ejemplo incorrecto: "1 millón de estudiantes, 500 mil concurrentes".

Criterios de Puntuación Detallada

20 puntos

- **Alumnos potenciales:** 10,000–50,000 (orden de magnitud universitario razonable)
- **Usuarios concurrentes:** 5–15 % de alumnos potenciales (ej: 500–7,500 en hora peak)
- **Disponibilidad:** 24x7 con justificación clara (sistema web con funciones asíncronas: agendamiento, cápsulas, comentarios, etc.)
- **Criticidad:** Media, bien justificada (importante pero no crítico; puede tener breves indisponibilidades sin comprometer procesos académicos esenciales)
- **Confidencialidad:** Alta con ejemplos concretos (RUT, correos, calificaciones, horarios, direcciones)
- **Seguridad:** Requerimientos específicos (autenticación, sesiones, protección de datos personales, encriptación)

Coherencia global: Los números son consistentes entre sí (ej: si hay 20,000 alumnos y 10 % concurrencia = 2,000 usuarios concurrentes es razonable)

15 puntos

- Incluye todos los elementos requeridos
- Justificaciones presentes pero menos detalladas
- Los números son coherentes pero los rangos pueden ser más amplios
- Menciona todos los aspectos sin profundizar tanto

10 puntos

- Falta algún aspecto importante (ej: no menciona confidencialidad)
- Justificaciones débiles o superficiales
- Números razonables pero sin explicar la relación entre ellos

- Menciona 4–5 de los 6 aspectos solicitados

5 puntos

- Incoherencia numérica grave (ej: 1 millón de estudiantes con 500,000 concurrentes)
- Supuestos contradictorios entre sí
- Falta de coherencia global
- Solo menciona 2–3 aspectos

0 puntos

- No responde
- Respuesta completamente incorrecta o fuera de contexto

6.b Tecnologías para el backend [20 pts]

Elementos clave que debe incluir un stack completo:

1. **Lenguaje de programación** (Node.js/JavaScript, Python, Java, Ruby, etc.)
2. **Framework web** (Express, Koa, Django, Flask, Spring Boot, Rails, etc.)
3. **Base de datos** (PostgreSQL, MySQL, MongoDB, etc.)
4. **ORM o herramienta de acceso a datos** (Sequelize, TypeORM, SQLAlchemy, Hibernate, etc.)
5. **Servidor web/middleware** (Nginx, Apache, o el servidor incluido en el framework)
6. **Sistema operativo/entorno** (Linux, Docker, contenedores)
7. **Seguridad** (bcrypt, JWT, HTTPS, políticas de CORS)

Distribución de puntaje:

20 puntos

- Incluye AL MENOS 5 de los 7 elementos mencionados
- Justifica la elección considerando: ecosistema, rapidez de desarrollo, soporte comunitario, escalabilidad, compatibilidad con los supuestos de la parte (a)
- Ejemplo aceptable: “Node.js + Express + PostgreSQL + Sequelize + Nginx + Ubuntu/Docker, porque permite desarrollo rápido, tiene gran comunidad, escala horizontalmente, y maneja bien las 2,000 conexiones concurrentes estimadas”
- La coherencia con los supuestos es evidente

15 puntos

- Incluye 4–5 elementos del stack

- Justificación presente pero genérica
- No relaciona explícitamente con los supuestos de la parte (a)

10 puntos

- Incluye 3–4 elementos
- Justificación muy básica o casi ausente
- Las tecnologías elegidas son apropiadas pero faltan componentes importantes

4 puntos

- Solo menciona 1–2 tecnologías
- Justificación muy débil o inexistente
- Falta de coherencia con los requerimientos

0 puntos

- Elección no pertinente (tecnologías obsoletas, no web, o completamente inapropiadas)
- No responde

Nota sobre JAMStack:

Si el estudiante propone JAMStack, debe explicar claramente:

- Cómo implementará las funcionalidades dinámicas (autenticación, gestión de tutorías)
- Qué servicios backend utilizará (APIs serverless, headless CMS, etc.)
- Por qué es apropiado para este caso de uso específico

La simple mención de JAMStack sin explicación detallada se califica como 4 puntos máximo.

6.c Framework de frontend basado en Python [20 pts]

Concepto clave a evaluar:

Comprensión de que los navegadores web ejecutan JavaScript nativamente, no Python. Esta es una limitación arquitectónica fundamental de la web.

Distribución de puntaje:

20 puntos

- Explica claramente que los navegadores solo ejecutan JavaScript de forma nativa
- Diferencia correctamente entre frontend (navegador) y backend (servidor)
- Menciona que Python se usa en el backend (Django, Flask) para generar HTML o APIs

- Puede mencionar alternativas emergentes con limitaciones:
 - PyScript/Pyodide (Python vía WebAssembly, aún inmaduro)
 - Brython (transpilador Python→JS, con limitaciones de rendimiento)
 - Frameworks que generan JavaScript desde Python (ej: Transcrypt)
- Concluye que la petición del jefe no es adecuada para producción actual
- Propone alternativa: Python en backend + framework JS moderno en frontend (React, Vue, etc.)

12 puntos

- Indica que “no es recomendable” o “no es lo habitual”
- Menciona que JavaScript es el lenguaje del frontend
- Explicación correcta pero sin tanto detalle técnico
- Puede no mencionar alternativas emergentes

8 puntos

- Menciona que hay dificultades pero sin explicar claramente el por qué
- Confusión entre frontend y backend
- Respuesta ambigua o poco clara

4 puntos

- Acepta la petición pero propone soluciones problemáticas
- Confunde completamente frontend con backend
- Menciona frameworks que no son relevantes o apropiados

0 puntos

- Acepta acríticamente sin ninguna explicación
- Nombra algo totalmente fuera de lugar (ej: “usar Python directamente en el HTML”)
- No demuestra comprensión de la arquitectura web
- No responde

Pregunta 7: React [90 pts]

7.a Conceptos fundamentales [40 pts]

1. JSX [10 pts]

Respuestas esperadas:

- **Qué es:** Extensión de sintaxis de JavaScript que parece HTML/XML
- **Por qué se usa:** Para describir la UI de forma declarativa dentro del código JavaScript
- **Cómo se transforma:** Se transpila (con Babel) a llamadas de `React.createElement()`

Distribución de puntaje:

- **10 pts:** Responde correctamente las 3 partes
- **7 pts:** Responde correctamente 2 de las 3 partes
- **4 pts:** Responde correctamente 1 de las 3 partes o respuestas muy superficiales
- **0 pts:** No responde o respuesta incorrecta

2. Props e inmutabilidad [10 pts]

Respuestas esperadas:

- Props son parámetros/atributos que el componente padre pasa al hijo
- Representan datos de entrada, son de solo lectura
- Se consideran inmutables para:
 - Mantener flujo de datos unidireccional (padre → hijo)
 - Hacer el comportamiento predecible y debuggeable
 - Facilitar el rastreo de cambios y optimizaciones de React

Distribución de puntaje:

- **10 pts:** Define props correctamente Y explica la inmutabilidad con al menos 2 razones
- **7 pts:** Define props correctamente Y menciona inmutabilidad con 1 razón
- **4 pts:** Define props correctamente PERO no explica bien la inmutabilidad
- **0 pts:** No responde o respuesta incorrecta

3. Hooks [10 pts]

Respuestas esperadas (aceptar cualquiera de estas explicaciones):

- Mecanismo para “engancharse” al ciclo de vida y sistema de React
- Permite vincular dinámicamente el DOM virtual de React con el DOM del navegador
- Permite a componentes funcionales manejar estado, efectos, contexto, etc.
- Funciones especiales que dan acceso a características de React sin usar clases

Distribución de puntaje:

- **10 pts:** Explicación clara y técnicamente correcta

- **7 pts:** Explicación correcta pero incompleta o poco precisa
- **4 pts:** Menciona algunos conceptos correctos pero confuso
- **0 pts:** No responde o respuesta incorrecta

4. useState y re-render [10 pts]

Respuestas esperadas:

- **Qué hace useState:**
 - Declara estado local en componente funcional
 - Retorna un par: [valor, función setter]
 - Mantiene vínculo entre variable y componente
 - El setter actualiza el valor y refleja cambios en la interfaz
- **Cuándo re-renderiza:**
 - Después de que termina el evento actual (clic, etc.)
 - En el próximo ciclo de render de React
 - De forma asíncrona (batch updates)
 - Cuando se llama al setter del estado

Distribución de puntaje:

- **10 pts:** Explica correctamente qué hace Y cuándo re-renderiza
- **7 pts:** Explica una parte correctamente, la otra incompleta
- **4 pts:** Explicación superficial de ambas partes
- **0 pts:** No responde o respuesta incorrecta

7.b Comunicación padre-hijo con CursoCard [50 pts]

Contexto:

Basado en el ejemplo visto en clases. Se espera que el estudiante:

1. Entienda que CursoCard es un componente reutilizable
2. Comprenda que ListaCursos renderiza múltiples instancias
3. Explique el flujo de datos bidireccional (props hacia abajo, callbacks hacia arriba)

Elementos clave que debe incluir la respuesta:

1. **Estructura de datos en el padre (ListaCursos):**
 - Array de objetos con información de cursos
 - Estado con useState para manejar favoritos
 - Ej: `const [cursos, setCursos] = useState([...])`

2. Renderizado dinámico con map:

- Iterar sobre el array de cursos
- Por cada curso, renderizar un `<CursoCard>`
- Pasar props: nombre, profesor, cupos

3. Callback para notificar al padre:

- El padre pasa una función como prop (ej: `onFavorito`)
- El hijo llama a esa función cuando se presiona el botón
- La función recibe algún identificador del curso

4. Actualización del estado en el padre:

- El padre actualiza su estado usando el setter de `useState`
- Marca el curso como favorito (modifica el array)

5. Re-render automático:

- React detecta el cambio de estado
- Re-renderiza `ListaCursos`
- Las tarjetas se actualizan, mostrando cuáles son favoritas
- Puede usar props o clases CSS condicionales para destacar

Distribución de puntaje:

50–45 puntos

- Describe claramente los 5 elementos clave
- Flujo completo y coherente de datos
- Menciona `useState` en el padre
- Explica el callback para comunicación hijo→padre
- Describe el re-render automático
- Puede incluir pseudocódigo o diagrama de flujo
- Demuestra comprensión profunda del patrón

44–35 puntos

- Describe 4 de los 5 elementos clave
- Flujo mayormente correcto con alguna imprecisión menor
- Puede faltar detalle en el callback o en el re-render
- La idea general es correcta

34–25 puntos

- Describe 3 de los 5 elementos clave
- Comprende la comunicación padre-hijo pero con confusiones
- Puede confundir dirección de datos
- Idea general presente pero incompleta

24–10 puntos

- Describe 1–2 elementos correctamente
- Confusión significativa en el flujo de datos
- No menciona useState o callbacks
- Muestra comprensión muy básica

9–0 puntos

- Respuesta mayormente incorrecta
- No comprende el patrón de comunicación
- No responde

Nota importante:

NO se evalúa sintaxis estricta. Se evalúa la comprensión del flujo lógico. El estudiante puede usar:

- Descripción en palabras
- Pseudocódigo
- Diagramas de flujo
- Mezcla de código y explicación

Pregunta 8: Vue.js [80 pts]

8.a beforeCreate vs created [40 pts]

Conceptos clave a evaluar:

1. Qué ocurre en beforeCreate:

- Se ejecuta antes de inicializar el sistema de reactividad
- La instancia del componente existe pero está “vacía”
- NO existen: data(), computed, methods, watchers
- NO hay acceso a `this.variable`

2. Qué ocurre en created:

- Ya se inicializó el sistema de reactividad
- Ya existen: data(), computed, methods
- Sí hay acceso a variables reactivas
- Todavía NO existe el DOM (no se ha montado)

3. Por qué no funciona en beforeCreate:

- Vue aún no ha procesado las opciones del componente
- El sistema de reactividad no está configurado
- Las variables declaradas en data() no existen aún
- Intentar acceder a ellas resulta en **undefined**

Distribución de puntaje:

40–35 puntos

- Explica correctamente qué inicializa Vue en cada fase
- Identifica claramente qué NO existe en beforeCreate
- Explica por qué no hay acceso a reactividad
- Menciona que el sistema de reactividad se configura entre beforeCreate y created
- Responde las 3 sub-preguntas de forma completa

34–25 puntos

- Explica la diferencia entre los dos hooks
- Identifica que las variables no existen en beforeCreate
- Responde 2 de las 3 sub-preguntas correctamente
- Puede faltar detalle sobre el sistema de reactividad

24–15 puntos

- Comprende que hay un orden en el ciclo de vida
- Menciona que las variables no están disponibles
- Responde 1 de las 3 sub-preguntas correctamente
- Explicación superficial o incompleta

14–0 puntos

- No comprende el ciclo de vida
- Confunde los hooks
- No responde o respuesta incorrecta

8.b Reloj en `onUpdated` vs `onMounted` [40 pts]

Conceptos clave a evaluar:

1. Por qué `onUpdated` es incorrecto (10 pts):

- `onUpdated` se ejecuta cada vez que el componente se actualiza
- Cada actualización crea un nuevo `setInterval`
- El reloj mismo causa actualizaciones (cada segundo)

2. Por qué el comportamiento observado (10 pts):

- Ciclo recursivo: actualización → `onUpdated` → nuevo `interval` → actualización
- Múltiples intervalos corriendo simultáneamente
- Cada `interval` actualiza la hora → más actualizaciones → más `intervals`
- Consumo exponencial de memoria y CPU
- Hora se actualiza de forma errática (múltiples actualizaciones/segundo)

3. Por qué `onMounted` es correcto (10 pts):

- Se ejecuta UNA SOLA VEZ después del montaje inicial
- El componente ya está en el DOM
- Las variables reactivas ya existen y están vinculadas
- Momento apropiado para configurar efectos secundarios persistentes

4. Comportamiento correcto con `onMounted` (10 pts):

- Se crea UN SOLO `interval` al montar el componente
- El `interval` actualiza la variable reactiva cada segundo
- La reactividad de Vue actualiza automáticamente el DOM
- No hay recursión ni múltiples `intervals`
- Se debe limpiar el `interval` en `onBeforeUnmount` para evitar memory leaks

Distribución de puntaje:

40–35 puntos

- Explica claramente los 4 puntos solicitados
- Identifica el ciclo recursivo en `onUpdated`
- Explica por qué se crean múltiples intervalos
- Justifica correctamente por qué `onMounted` es apropiado
- Describe el comportamiento correcto del reloj
- Puede mencionar la limpieza del `interval` (bonus conceptual)

34–25 puntos

- Explica 3 de los 4 puntos correctamente

- Identifica que `onUpdated` causa problemas
- Comprende que `onMounted` es la solución
- Puede faltar detalle en el ciclo recursivo o comportamiento correcto

24–15 puntos

- Explica 2 de los 4 puntos
- Comprende que hay un problema con `onUpdated`
- Mención básica de `onMounted` como solución
- Explicación superficial del por qué

14–0 puntos

- Explica solo 1 punto o ninguno correctamente
- No comprende el problema del ciclo recursivo
- Confunde los hooks del ciclo de vida
- No responde o respuesta incorrecta

Nota Final Importante

Recordatorios para el corrector:

- La pregunta 7b) está basada en un ejemplo visto en clases - ser flexible con la forma de expresión
- NO evaluar sintaxis estricta de código, sino comprensión de conceptos
- Valorar explicaciones claras aunque usen notación informal
- En caso de duda entre dos puntajes, considerar la coherencia global de la respuesta
- Si el estudiante demuestra comprensión del concepto pero con errores menores, no penalizar severamente

Bonus: Preguntas Opcionales Valor: Cada pregunta bonus correcta otorga 0.5 décimas adicionales (máximo 1.0 en la nota final).

Pregunta 9: Bonus 1: React (useEffect + Virtual DOM) [50 pts]

Conceptos fundamentales [20 pts]

1. Virtual DOM (VDOM) [5 pts]:

- **Propósito:** Una representación ligera en memoria (un árbol de objetos JavaScript) del DOM real.
- **Función:** Permite a React calcular las diferencias (diffing) entre el estado actual y el nuevo estado.
- **Por qué no actualiza el DOM real inmediatamente:** El DOM real es lento. React actualiza el VDOM, calcula un **patch** con las diferencias mínimas necesarias, y luego actualiza el DOM real de forma eficiente.

2. Estrategia Memoization [5 pts]:

- **Qué significa:** Es una técnica de optimización donde el resultado de una función costosa se almacena en caché.
- **Propósito en React:** Se usa en `React.memo`, `useMemo` y `useCallback` para evitar re-renderizados o re-cálculos innecesarios de componentes/valores/funciones si sus **props** o **dependencias** no han cambiado.

3. Propósito de un hook [5 pts]:

- Permite a las **funciones de componente** “engancharse” a características de React (estado, ciclo de vida, contexto).
- Permite reutilizar lógica de estado entre componentes.

4. useEffect [5 pts]:

- **Qué hace:** Ejecuta código con “efectos secundarios” (side effects) después de que React ha renderizado el componente.
- **Cuándo se ejecuta:** Después del primer render Y después de cada re-render subsiguiente si alguna de sus dependencias ha cambiado.

9.a Estructura de useEffect para Fetch [15 pts]

1. Dependencias [5 pts]:

- **Dependencia(s) ideal(es):** Arreglo vacío (`[]`)
- **Por qué:** Garantiza que el efecto se ejecute **una sola vez** al montarse el componente, como un `componentDidMount` (evitando re-fetches en cada render).

2. Riesgo de omitir el arreglo [5 pts]:

- **Riesgo:** El efecto se ejecutaría después de **cada** renderizado del componente. Si el componente se re-renderiza con frecuencia (ej: por cambios de estado), el **fetch** ocurriría

repetidamente, saturando el servidor.

3. Riesgo de dependencias incorrectas [5 pts]:

- **Riesgo:** Si la dependencia es un estado que se actualiza dentro del mismo `useEffect` (como en la parte c), se crea un **bucle infinito** de re-renders y re-ejecuciones.

9.b Análisis de bucle infinito (Pseudocódigo) [15 pts]

Pasos del ciclo infinito:

1. **Inicio:** El componente se renderiza por primera vez.
2. **Ejecución de `useEffect`:** Se ejecuta el código interno, haciendo el `fetch`.
3. **Actualización de estado:** La promesa se resuelve y llama a `setCursos(data)`.
4. **Re-render:** `useState` detecta un cambio. React vuelve a renderizar el componente.
5. **Detección de dependencia:** Al finalizar el re-render, `useEffect` compara la dependencia [`cursos`] con su valor anterior. Como `setCursos` cambió el valor de `cursos`, React detecta el cambio.
6. **Nuevo ciclo:** Debido al cambio en `cursos`, `useEffect` se ejecuta de nuevo (paso 2), iniciando un nuevo `fetch` y un nuevo `setCursos`, y repitiendo el ciclo indefinidamente.

Puntuación:

- **15 pts:** Explica el ciclo de renders, identificando la relación `useEffect` → `setCursos` → `cursos` → `useEffect`.
- **8 pts:** Identifica el bucle, pero sin relacionar explícitamente `useState` y la detección de dependencias.

Pregunta 10: Bonus 2: Vue 3 (ref + fetch + reactividad) [50 pts]

10.a Conceptos de Reactividad [20 pts]

1. `ref()` y `.value` [10 pts]:

- **Qué es `ref()`:** Una función que permite declarar una **variable reactiva** primitiva (o un objeto) que mantendrá su valor a través de los ciclos de render.
- **Por qué `.value`:** `ref` crea un objeto contenedor que envuelve el valor. Vue necesita acceder a esta propiedad `.value` para **rastrear** y **“engancharse”** su reactividad, y para poder re-asignarlo correctamente.
- **Excepción:** Cuando se usa en la plantilla (`<template>`), Vue automáticamente “des-envuelve” (*unwraps*) el `.value`, permitiendo el acceso directo.

2. Conversión a Objeto Reactivo y DOM [10 pts]:

- **Cómo convierte Vue:** Utiliza un sistema de **observadores** (`watchers`) y **proxies**

(en el caso de `reactive`) que detectan cuando se accede o se modifica el valor dentro del `.value` del `ref`.

- **Actualización del DOM:** Al detectarse un cambio en el valor reactivo, Vue notifica a los componentes que dependen de esa variable. Estos componentes se vuelven a renderizar, y Vue actualiza el DOM real de forma eficiente (similar a React) utilizando el **Virtual DOM**.
- **Eventos del ciclo de vida:** El hook `onUpdated` se dispara después de que la actualización del DOM ha ocurrido debido a un cambio de estado reactivo.

10.b Proceso de carga de datos [20 pts]

Diseño de variables y hook [10 pts]:

- **Variables con `ref`** (mínimo 2):
 - `const asignaturas = ref([]);` (Guardar datos)
 - `const cargando = ref(false);` (Estado de carga)
 - `const error = ref(null);` (Manejo de errores)
- **Hook correcto:** `onMounted()` (Se ejecuta UNA SOLA VEZ, al igual que `useEffect` con `[]`).

Flujo de Reactividad [10 pts]:

1. En `onMounted()`, se llama a `cargando.value = true`.
2. Vue detecta el cambio en `cargando.value` y automáticamente actualiza la plantilla (*template*) (ej: mostrando un spinner).
3. Se ejecuta el `fetch`.
4. Al recibir la respuesta exitosa, se llama a `asignaturas.value = data`.
5. Vue detecta el cambio en `asignaturas.value` y actualiza la plantilla (muestra la lista).
6. Finalmente, se llama a `cargando.value = false` y la plantilla oculta el spinner.

10.c Análisis de falla [10 pts]

¿Por qué falla? [5 pts]:

- La línea `asignaturas = data` **re-asigna** la variable local `asignaturas` a un nuevo valor de `data`.
- Esta re-asignación rompe el vínculo que Vue había creado entre el nombre de la variable y el objeto `ref()` reactivo.
- El valor dentro del contenedor original nunca se actualizó, por lo tanto, el sistema de reactividad no detecta el cambio y el DOM no se actualiza.

Forma correcta [5 pts]:

- Se debe actualizar el valor de la propiedad `.value` del `ref` para que Vue pueda rastrear el cambio:

- `asignaturas.value = data;`

Pregunta 11: Bonus 3: ORM [50 pts]

11.a Conceptos de ORM y Mapeo [25 pts]

1. ORM y su rol [10 pts]:

- **Qué es:** Object-Relational Mapper (Mapeador Objeto-Relacional).
- **Rol:** Actúa como un puente entre la programación orientada a objetos (clases, objetos) y las bases de datos relacionales (tablas, filas).
- **Función:** Permite interactuar con la BDD usando métodos de POO (ej: `Estudiante.create()`) en lugar de escribir consultas SQL directamente.

2. Mapeo en Sequelize [5 pts]:

- El modelo (`Estudiante`) define el **esquema** y la **estructura** de la tabla en la BDD, y al mismo tiempo, permite manipular las filas de esa tabla como instancias de una **clase** de JavaScript.
- Esto permite que las filas de la tabla `estudiantes` se conviertan en objetos de la clase `Estudiante` y viceversa.

3. Diferencia conceptual [10 pts]:

- **define():** Se usa para **definir la estructura** (el esquema) del modelo en la memoria del ORM, antes de interactuar con la BDD.
- **sync():** Se usa para **sincronizar** el esquema definido del modelo con el **esquema real** de la base de datos (crea o modifica la tabla).
- **create():** Método de instancia que se usa para **insertar una nueva fila** de datos en la tabla (ejecuta un `INSERT INTO`).
- **findOne():** Método de clase que se usa para **recuperar una única fila** de datos de la tabla (ejecuta un `SELECT * LIMIT 1`).

11.b Flujo arquitectónico de creación [10 pts]

Flujo esperado (Arquitectura de 3 capas):

1. **Controlador** (Controlador Estudiante): Recibe la solicitud HTTP (`POST /estudiantes`) y extrae los datos del cuerpo de la solicitud (`body`).
2. **Capa de Servicio** (Servicio Estudiante): El controlador llama a una función en la capa de servicio (ej: `EstudianteService.crear(datos)`).
3. **Validación:** La capa de servicio se encarga de las **validaciones de dominio** (ej: asegurar que el RUT tenga el formato correcto, o que el estudiante sea mayor de edad).
4. **Modelo:** El servicio utiliza el modelo de Sequelize (ej: `Estudiante.create(datos)`).

para ejecutar la inserción en la BDD.

5. Manejo de Errores:

- El servicio maneja errores de dominio (ej: si la validación falla).
- El controlador maneja la respuesta del servicio y los errores que puedan haber ocurrido a nivel de BDD o sistema, y genera la respuesta HTTP (201 `Created` o 4xx/5xx).

11.c Errores de sincronización [15 pts]

¿Por qué ocurren los errores? [5 pts]:

- El ORM está definido en el código con el nuevo campo `email`, pero la **tabla física** en la base de datos aún no tiene esa columna.
- Los métodos de consulta (`findAll`, etc.) intentan generar SQL que hace referencia a una columna inexistente, lo que genera los errores.
- **Concepto involucrado:** El esquema en memoria (código) está desfasado con el esquema de la base de datos.

Mecanismos de corrección [5 pts]:

- El equipo debe usar **Mecanismos de Sincronización o Migraciones**.
- **Sincronización Simple:** Ejecutar `sequelize.sync()` (solo crea la tabla si no existe, no añade columnas automáticamente a menos que se use `alter: true`).
- **Migraciones (Método ideal):** Generar un archivo de migración que ejecute una consulta `ALTER TABLE` para añadir específicamente la columna `email` sin tocar los datos existentes.

Riesgos de `force: true` [5 pts]:

- La opción `force: true` en `sequelize.sync()` borra (ejecuta `DROP TABLE`) y recrea la tabla en cada inicio.
- **Riesgo:** Pérdida irreversible de **todos los datos** contenidos en la tabla `Estudiante`, ya que la tabla se elimina por completo.
- **Alternativa segura:** Usar `migrations` es la práctica estándar en producción.